

# llmsverifier\_analysis

## LLMsVerifier Repository — Deep Analysis Report

**Repository:** <https://github.com/vasic-digital/LLMsVerifier>  
**Branch:** main  
**Primary Language:** Go (75.3%)  
**Analysis Date:** 2026  
**Total Commits:** ~519  
**Contributors:** milos85vasic, claude

---

### 1. Repository Structure

#### Root-Level Directories

| Directory       | Purpose                                                                                     |
|-----------------|---------------------------------------------------------------------------------------------|
| llm-verifier/   | <b>Main Go application</b> — core engine, CLI, API server, providers, scoring, verification |
| internal/       | Internal packages: benchmark/, llmops/, messaging/, rag/, selfimprove/                      |
| configs/        | Configuration files and schemas                                                             |
| docs/           | Documentation (governance, architecture, design docs)                                       |
| examples/       | Usage examples (e.g., examples/scoring/)                                                    |
| tests/          | Test suites (unit, integration, e2e, performance, security)                                 |
| challenges/     | Challenge system scripts and specifications                                                 |
| sdk/            | SDK for client integrations                                                                 |
| helm/           | Helm charts for Kubernetes deployment                                                       |
| k8s/            | Kubernetes manifests                                                                        |
| monitoring/     | Monitoring and observability configs                                                        |
| website/        | Website source                                                                              |
| mobile/flutter/ | Mobile application (Flutter)                                                                |
| assets/         | Logo and visual assets                                                                      |

| Directory     | Purpose                                         |
|---------------|-------------------------------------------------|
| upstreams/    | Upstream repository management                  |
| reports/      | Generated reports storage                       |
| scripts/      | Utility scripts                                 |
| specs/        | Specifications (e.g., 001-extend-llm-providers) |
| test_results/ | Test result outputs                             |
| video-course/ | Video course assets                             |
| backup/       | Backup data                                     |

## llm-verifier/ Subdirectory Structure

```

llm-verifier/
  cmd/main.go          # CLI entry point (~1633 lines)
  ai/                  # AI-related modules
  analytics/           # Analytics and metrics
  api/                 # REST API server (gin-based)
  api_keys/            # API key tracking and management
  auth/                # OAuth, JWT, LDAP authentication
  bigdata/             # Big Data integration
  capabilities/        # Capability detection
  challenges/          # Challenge system
  client/              # HTTP client for API communication
  config/              # Configuration structs and loading
  database/            # SQLite persistence layer
  desktop/             # Desktop application code
  docs/               # Package-level docs
  enhanced/           # Enhanced features
  events/              # Event system
  failover/           # Failover logic
  k8s/                # K8s-specific code
  llmverifier/         # Core verification engine
    recipes/           # Verification recipes
    analytics.go
    config_loader.go
    config_export.go
    issue_detector.go
    llm_client.go      # LLM HTTP client
    models.go          # Data models
    reporter.go        # Report generation
    strategy.go        # Scoring strategy interface
    strategy_builder.go
    strategy_default.go
  logging/             # Logging infrastructure
  messaging/           # Kafka/RabbitMQ integration
  mobile/              # Mobile app helpers
  monitoring/          # Monitoring hooks
  multimodal/          # Multimodal capabilities
  notifications/       # Slack, Email, Telegram, Matrix, WhatsApp

```

|                                   |                                    |
|-----------------------------------|------------------------------------|
| partners/                         | # Partner integrations             |
| performance/                      | # Performance testing              |
| pkg/                              | # Shared packages                  |
| providers/                        | # Provider adapters                |
| base.go                           | # BaseAdapter struct               |
| anthropic.go                      | # Anthropic (Claude) adapter       |
| cerebras.go                       |                                    |
| cloudflare.go                     |                                    |
| cohere.go                         |                                    |
| deepseek.go                       |                                    |
| groq.go                           |                                    |
| hyperbolic.go                     |                                    |
| kilo.go                           |                                    |
| kimim.go                          |                                    |
| kimicode.go                       |                                    |
| mistral.go                        |                                    |
| modal.go                          |                                    |
| model_provider_service.go         |                                    |
| errors.go                         |                                    |
| fallback_models.go                |                                    |
| http_client.go                    |                                    |
| config.go                         |                                    |
| config_validator.go               |                                    |
| scheduler/                        | # Verification scheduling          |
| scoring/                          | # Scoring system                   |
| scoring_engine.go                 | # Core scoring logic               |
| types.go                          | # Score data structures            |
| metrics_collector.go              |                                    |
| alert_manager.go                  |                                    |
| api_handlers.go                   |                                    |
| database_integration.go           |                                    |
| model_display.go                  |                                    |
| model_naming.go                   |                                    |
| monitoring.go                     |                                    |
| scripts/                          | # Build and deploy scripts         |
| sdk/                              | # SDK packages                     |
| security/                         | # Security modules                 |
| suffix/                           | # Suffix handling                  |
| test_exports/                     | # Test exports                     |
| testing/                          | # Testing utilities                |
| testsuite/                        | # Test suite runner                |
| tui/                              | # Terminal UI (bubbletea)          |
| verification/                     | # Verification logic               |
| verification.go                   | # Main verifier (stub/placeholder) |
| code_verification.go              |                                    |
| code_verification_integration.go  |                                    |
| coding_capability_verification.go |                                    |
| models_dev_enhanced.go            |                                    |
| provider_client.go                |                                    |
| provider_service_interface.go     |                                    |
| verification_real.go              |                                    |
| web/                              | # Web UI components                |

---

## 2. README & Documentation

File: `llm-verifier/README.md` (400 lines) [<sup>18</sup>]

### Purpose

“LLM Verifier is a comprehensive tool to verify, test, and benchmark LLMs based on their coding capabilities and other features.”

### Supported Providers (12 Implemented Adapters)

1. **OpenAI** — GPT models with full API compatibility
2. **Anthropic** — Claude models via official API
3. **DeepSeek** — DeepSeek models with streaming support
4. **Groq** — Fast inference with Llama models
5. **Together AI** — Wide range of open-source models
6. **Mistral** — European provider with advanced models
7. **xAI** — Grok models from xAI
8. **Replicate** — Model hosting and deployment platform
9. **Cohere** — Command models with enterprise features
10. **Cerebras** — High-performance inference
11. **Cloudflare Workers AI** — Edge AI inference
12. **SiliconFlow** — Chinese AI models and services

### Features

- Model Discovery (auto-discover from API endpoints)
- Comprehensive Testing (existence, responsiveness, overload, capabilities)
- Feature Detection (tool calling, embeddings, code generation, etc.)
- Coding Assessment (multiple programming languages)
- Performance Scoring (code capability, responsiveness, reliability, feature richness)
- Reporting (markdown + JSON)
- Rankings (by strength, speed, reliability, etc.)

### CLI Commands

```
llm-verifier models list [--filter NAME] [--limit N] [--format
    json|table]
llm-verifier models get MODEL_ID
llm-verifier models create PROVIDER_ID MODEL_ID NAME
llm-verifier models verify MODEL_ID
llm-
    verifier providers list [--filter NAME] [--limit N] [--
    format json|table]
llm-verifier results list [--filter MODEL_NAME] [--limit N] [--
    format json|table]
llm-verifier pricing list [--format json|table]
llm-verifier limits list [--format json|table]
llm-verifier issues list [--format json|table]
llm-verifier events list [--format json|table]
```

```

llm-verifier schedules list [--format json|table]
llm-verifier exports list [--format json|table]
llm-verifier logs list [--format json|table]
llm-verifier config show
llm-verifier config export FORMAT
llm-verifier users create USERNAME PASSWORD EMAIL [FULL_NAME]
llm-verifier tui                                # Terminal UI
llm-verifier server [--port PORT]              # REST API server
llm-verifier ai-config export [format] [output_file]
llm-verifier ai-config bulk [output_directory]

```

## Scoring System Weights (from README)

- **Code Capability (40%)**
  - **Responsiveness (20%)**
  - **Reliability (20%)**
  - **Feature Richness (15%)**
  - **Value Proposition (5%)**
- 

## 3. Core Verification Engine

### Entry Point

**File:** `llm-verifier/cmd/main.go` (line 84-106) <sup>[19]</sup>

```

func runVerification() error {
    cfg, err := llmverifier.LoadConfig(configFile)
    verifier := llmverifier.New(cfg)
    results, err := verifier.Verify()
    verifier.GenerateMarkdownReport(results, outputDir)
    verifier.GenerateJSONReport(results, outputDir)
}

```

### Main Verifier

**File:** `llm-verifier/verification/verification.go` (130 lines) <sup>[28]</sup>

**CRITICAL FINDING:** The `core verification.go` is currently a **stub/placeholder implementation** that returns hardcoded scores:

```

func (v *Verifier) Verify(ctx context.Context, req *Request)
    (*database.VerificationResult, error) {
    // ... validation ...
    result := &database.VerificationResult{
        ID: time.Now().UnixNano(),
        ModelID: 1, // Placeholder
        Status: "completed",
        ModelExists: boolPtr(true),
    }
}

```

```

        Responsive: boolPtr(true),
        Overloaded: boolPtr(false),
        SupportsToolUse: true,
        SupportsCodeGeneration: true,
        // ... ALL fields hardcoded to true or 8.5 ...
        OverallScore: 8.5,
        CodeCapabilityScore: 8.5,
        ResponsivenessScore: 8.5,
        ReliabilityScore: 8.5,
        FeatureRichnessScore: 8.5,
        ValuePropositionScore: 8.5,
    }
    return result, nil
}

```

**NOTE:** The actual verification logic for coding capabilities appears to be in `verification/coding_capability_verification.go` which performs real LLM API calls with keyword matching.

## Coding Capability Verification

**File:** `llm-verifier/verification/coding_capability_verification.go`  
(~450 lines) <sup>[28]</sup>

- Tests 4 dimensions: **Codebase Detection**, **Language Detection**, **Code Generation**, **Code Analysis**
- Uses keyword matching against expected terms
- Threshold:  $\geq 0.6$  readiness score for “ReadyForCoding”
- Status mapping:  $\geq 0.7 \rightarrow$  “verified”,  $\geq 0.4 \rightarrow$  “partial”, else “failed”
- Makes real LLM API calls via `ProviderClientInterface`

---

## 4. Provider Support

### Base Adapter

**File:** `llm-verifier/providers/base.go` (62 lines) <sup>[27]</sup>

```

type BaseAdapter struct {
    client    *http.Client
    endpoint  string
    apiKey    string
    headers   map[string]string
}

```

### Provider Files in `providers/`

| File                      | Provider           | Auth            |
|---------------------------|--------------------|-----------------|
| <code>anthropic.go</code> | Anthropic (Claude) | API Key + OAuth |

| File          | Provider              | Auth    |
|---------------|-----------------------|---------|
| deepseek.go   | DeepSeek              | API Key |
| groq.go       | Groq                  | API Key |
| cerebras.go   | Cerebras              | API Key |
| cloudflare.go | Cloudflare Workers AI | API Key |
| cohere.go     | Cohere                | API Key |
| hyperbolic.go | Hyperbolic            | API Key |
| kilo.go       | Kilo                  | API Key |
| kimi.go       | Kimi                  | API Key |
| kimicode.go   | Kimi Code             | API Key |
| mistral.go    | Mistral               | API Key |
| modal.go      | Modal                 | API Key |

**File:** providers/anthropic.go (excerpt, lines 26-78) <sup>[27]</sup>

```
type AnthropicAdapter struct {
    BaseAdapter
    authType      AuthType
    oauthCredReader *auth.OAuthCredentialReader
}
// Supports both API Key (x-api-key) and OAuth (Bearer token)
// Header: "anthropic-version": "2023-06-01"
```

## Provider Configuration

**File:** providers/config.go — defines provider metadata, endpoints, rate limits **File:** providers/fallback\_models.go — fallback model lists when APIs fail **File:** providers/errors.go — provider-specific error types (ProviderInitError, etc.)

## How New Providers Are Added

1. Create a new file in providers/ (e.g., newprovider.go)
2. Embed BaseAdapter
3. Implement provider-specific request/response conversion
4. Add to provider registry/service
5. Add fallback models to fallback\_models.go

---

## 5. Model Management

### Model Data Structure

**File:** llm-verifier/llmverifier/models.go (excerpt, lines 6-55) <sup>[40]</sup>

```

type VerificationResult struct {
    ModelInfo          ModelInfo
        `json:"model_info"`
    Availability        AvailabilityResult
        `json:"availability"`
    ResponseTime        ResponseTimeResult
        `json:"response_time"`
    FeatureDetection    FeatureDetectionResult
        `json:"feature_detection"`
    CodeCapabilities    CodeCapabilityResult
        `json:"code_capabilities"`
    GenerativeCapabilities GenerativeCapabilityResult
        `json:"generative_capabilities,omitempty"`
    PerformanceScores   PerformanceScore
        `json:"performance_scores"`
    Timestamp           time.Time
        `json:"timestamp"`
    Error               string
        `json:"error,omitempty"`
    ScoreDetails        ScoreDetails
        `json:"score_details"`
}

```

```

type ModelInfo struct {
    ID                string        `json:"id"`
    Object            string        `json:"object"`
    Created           int64         `json:"created"`
    OwnedBy          string        `json:"owned_by"`
    Root              string        `json:"root,omitempty"`
    Parent            string        `json:"parent,omitempty"`
    Permissions       []Permission
        `json:"permissions,omitempty"`
    ScalingPolicy     *ScalingPolicy
        `json:"scaling_policy,omitempty"`
    Capabilities      Capabilities
        `json:"capabilities,omitempty"`
    ContextWindow     ContextWindow
        `json:"context_window,omitempty"`
    MaxOutputTokens   int
        `json:"max_output_tokens,omitempty"`
    InputPrices        InputPrices
        `json:"input_prices,omitempty"`
    OutputPrices       OutputPrices
        `json:"output_prices,omitempty"`
    HasTrainingData   bool
        `json:"has_training_data,omitempty"`
    Description        string
        `json:"description,omitempty"`
}

```



```

Architecture      Architecture
    `json:"architecture,omitempty"`
Tokenizer          string      `json:"tokenizer,omitempty"`
Organization       string
    `json:"organization,omitempty"`
ReleaseDate        string
    `json:"release_date,omitempty"`
LanguageSupport    []string
    `json:"language_support,omitempty"`
UseCase            string      `json:"use_case,omitempty"`
Version            string      `json:"version,omitempty"`
MaxInputTokens     int
    `json:"max_input_tokens,omitempty"`
SupportsVision     bool
    `json:"supports_vision,omitempty"`
SupportsAudio      bool
    `json:"supports_audio,omitempty"`
SupportsVideo      bool
    `json:"supports_video,omitempty"`
SupportsReasoning  bool
    `json:"supports_reasoning,omitempty"`
SupportsHTTP3      bool
    `json:"supports_http3,omitempty"`
SupportsToon       bool
    `json:"supports_toon,omitempty"`
SupportsBrotli     bool
    `json:"supports_brotli,omitempty"`
OpenSource         bool
    `json:"open_source,omitempty"`
Deprecated         bool
    `json:"deprecated,omitempty"`
Tags               []string  `json:"tags,omitempty"`
Endpoint           string    `json:"endpoint"`
}

```

## Database Model Schema

**File:** llm-verifier/database/database.go (lines 444-479) <sup>[28]</sup>

```

CREATE TABLE IF NOT EXISTS models (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    provider_id INTEGER NOT NULL,
    model_id TEXT NOT NULL,
    name TEXT NOT NULL,
    description TEXT,
    version TEXT,
    architecture TEXT,
    parameter_count INTEGER,

```

```

context_window_tokens INTEGER,
max_output_tokens INTEGER,
training_data_cutoff DATE,
release_date DATE,
is_multimodal BOOLEAN DEFAULT 0,
supports_vision BOOLEAN DEFAULT 0,
supports_audio BOOLEAN DEFAULT 0,
supports_video BOOLEAN DEFAULT 0,
supports_reasoning BOOLEAN DEFAULT 0,
open_source BOOLEAN DEFAULT 0,
deprecated BOOLEAN DEFAULT 0,
tags TEXT,
language_support TEXT,
use_case TEXT,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
last_verified TIMESTAMP,
verification_status TEXT DEFAULT 'pending',
overall_score REAL DEFAULT 0.0,
code_capability_score REAL DEFAULT 0.0,
responsiveness_score REAL DEFAULT 0.0,
reliability_score REAL DEFAULT 0.0,
feature_richness_score REAL DEFAULT 0.0,
value_proposition_score REAL DEFAULT 0.0,
FOREIGN KEY (provider_id) REFERENCES providers(id) ON DELETE
    CASCADE
);

```

## Model Discovery

- `llmverifier/llm_client.go` — `ListModels()` calls `/models` endpoint on provider
- Returns `[]ModelInfo` with full metadata

---

## 6. Rate Limiting & Cooldown

### Configuration

**File:** `llm-verifier/config/config.go` (lines 139-158) <sup>[28]</sup>

```

type APIConfig struct {
    Port          string `mapstructure:"port"`
    JWTSecret     string `mapstructure:"jwt_secret"`
    RateLimit     int    `mapstructure:"rate_limit" // requests per
                        minute
    BurstLimit    int    `mapstructure:"burst_limit"`
}

```

```

    RateLimitWindow    int
        `mapstructure:"rate_limit_window"`      // seconds
    EnableCORS          bool    `mapstructure:"enable_cors"`
    RateLimitByAPIKey   bool
        `mapstructure:"rate_limit_by_api_key"`
    // ... TLS, timeouts ...
}

```

## Database Limits Table

File: llm-verifier/database/database.go (lines 499-512)

```

CREATE TABLE IF NOT EXISTS limits (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    model_id INTEGER NOT NULL,
    limit_type TEXT NOT NULL,
    limit_value INTEGER NOT NULL,
    current_usage INTEGER DEFAULT 0,
    reset_period TEXT,
    reset_time TIMESTAMP,
    is_hard_limit BOOLEAN DEFAULT 1,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (model_id) REFERENCES models(id) ON DELETE
        CASCADE
);

```

## Provider-Level Rate Limiting

File: llm-verifier/providers/config.go — contains ProviderConfig with RateLimit, RateLimitWindow, RetryPolicy

## Global Request Delay

File: llm-verifier/config.yaml (line 14): request\_delay: 1s

---

# 7. Token Tracking & Pricing

## Pricing Table

File: llm-verifier/database/database.go (lines 481-497)

```

CREATE TABLE IF NOT EXISTS pricing (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    model_id INTEGER NOT NULL,
    input_token_cost REAL DEFAULT 0.0,
    output_token_cost REAL DEFAULT 0.0,
    cached_input_token_cost REAL DEFAULT 0.0,

```

```

storage_cost REAL DEFAULT 0.0,
request_cost REAL DEFAULT 0.0,
currency TEXT DEFAULT 'USD',
pricing_model TEXT DEFAULT 'per_token',
effective_from DATE,
effective_to DATE,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (model_id) REFERENCES models(id) ON DELETE
    CASCADE
);

```

## Token Usage Tracking

**File:** llm-verifier/llmverifier/llm\_client.go (lines 114-119)

```

type Usage struct {
    PromptTokens      int `json:"prompt_tokens"`
    CompletionTokens  int `json:"completion_tokens"`
    TotalTokens       int `json:"total_tokens"`
}

```

## Cost Effectiveness Scoring

**File:** llm-verifier/scoring/scoring\_engine.go (lines 223-264) <sup>[41]</sup>

- Input token cost < 1.0 → +2.0 score
- Input token cost < 5.0 → +1.0 score
- Input token cost > 15.0 → -2.0 score
- Open source models → +2.0 score

---

## 8. Real-time Updates

**Finding:** The repository does **not** appear to have a webhook or push-based update system. Updates appear to be:

1. **Polling-based:** The TUI auto-refreshes every 30-60 seconds
2. **CLI-triggered:** llm-verifier models verify MODEL\_ID initiates verification on-demand
3. **Scheduled:** llm-verifier schedules list suggests scheduled verifications exist
4. **Manual refresh:** TUI supports r / R key for manual refresh

**File:** llm-verifier/cmd/main.go (line 63-66): schedulesCmd() for schedule management

---

## 9. API / Interface

### REST API Server

File: `llm-verifier/api/server.go` (70 lines, currently minimal) <sup>[28]</sup>

```
type Server struct {
    config    *config.Config
    database  *database.Database
    server    *http.Server
}

// Endpoints registered:
//   /api/health      -> HealthHandler
//   /api/models      -> ListModelsHandler
//   /api/models/{id} -> GetModelHandler
//   /api/models/{id}/verify -> VerifyModelHandler
//   /api/providers   -> ProvidersHandler
```

**CRITICAL GAP:** The `api/server.go` is **extremely minimal** — only 5 endpoints. The full API surface is much larger (models CRUD, providers, results, pricing, limits, issues, events, schedules, exports, logs, config, users, batch, tui). These likely exist in other files under `api/` (e.g., `api/handlers.go`, `api/routes.go`) but the current `server.go` only wires a basic mux.

### CLI Client

File: `llm-verifier/client/client.go` — HTTP client that communicates with the REST API server

### Data Export Formats

- **Markdown:** `llm_verification_report.md`
- **JSON:** `llm_verification_report.json`
- **AI CLI formats:** `opencode`, `crush`, `claude-code` configurations

---

## 10. Configuration System

### Configuration File

File: `llm-verifier/config.yaml` (18 lines) <sup>[17]</sup>

```
api:
  enable_cors: true
  jwt_secret: your-secret-key-change-in-production
  port: '8080'
  rate_limit: 100
concurrency: 5
```

```

database:
  encryption_key: ''
  path: ./llm-verifier.db
global:
  api_key: ${OPENAI_API_KEY}
  base_url: https://api.openai.com/v1
  max_retries: 3
  request_delay: 1s
  timeout: 30s
llms: []
timeout: 60s

```

## Config Struct

**File:** llm-verifier/config/config.go (lines 97-110) <sup>[28]</sup>

```

type Config struct {
    Profile      string                `mapstructure:"profile"`
    LLMs         []LLMConfig           `mapstructure:"llms"`
    Global       GlobalConfig           `mapstructure:"global"`
    Database     DatabaseConfig         `mapstructure:"database"`
    API          APIConfig              `mapstructure:"api"`
    Concurrency  int                   `mapstructure:"concurrency"`
    Timeout      time.Duration          `mapstructure:"timeout"`
    Logging      LoggingConfig           `mapstructure:"logging"`
    Monitoring   MonitoringConfig        `mapstructure:"monitoring"`
    Security     SecurityConfig          `mapstructure:"security"`
    Notifications NotificationsConfig     `mapstructure:"notifications"`
}

```

## Loading

- `config.LoadFromFile(path)` — supports YAML, JSON, TOML via Viper
- Environment variable substitution: `${OPENAI_API_KEY}`

## Full Configuration Example

**File:** llm-verifier/config\_full.yaml — comprehensive example with all providers, features, and options

---

# 11. Data Storage

## SQLite with Optional Encryption

File: `llm-verifier/database/database.go` (1661 lines) [<sup>28</sup>]

- **Driver:** `github.com/mattn/go-sqlite3`
- **Encryption:** SQL Cipher (optional, via `_pragma_key`)
- **WAL mode:** Enabled for concurrency
- **Connection pooling:** Max 25 open, 5 idle, 5min lifetime
- **Migrations:** Built-in migration manager with version tracking

### Key Tables

1. `users` — Authentication
  2. `api_keys` — Programmatic access keys
  3. `providers` — Provider metadata and status
  4. `models` — Model definitions and scores
  5. `pricing` — Token pricing per model
  6. `limits` — Rate limits per model
  7. `verification_results` — Full verification outputs
  8. `issues` — Detected issues and workarounds
  9. `events` — System events
  10. `schedules` — Verification schedules
  11. `exports` — Configuration exports
  12. `logs` — System logs
  13. `scoring_events` — Scoring history
- 

# 12. Validation Results Structure

## VerificationResult (Database)

File: `llm-verifier/database/database.go` (lines 514-581)

Key fields: - `id`, `model_id`, `verification_type` - `started_at`, `completed_at`, `status` - `model_exists`, `responsive`, `overloaded`, `latency_ms` - Feature flags: `supports_tool_use`, `supports_code_generation`, `supports_embeddings`, `supports_streaming`, `supports_json_mode`, `supports_reasoning`, `supports_parallel_tool_use`, `supports_batch_processing`, `supports_brotli` - Code capabilities: `code_debugging`, `code_optimization`, `test_generation`, `documentation_generation`, `refactoring`, `error_resolution`, `architecture_design`, `security_assessment`, `pattern_recognition` - Scores: `debugging_accuracy`, `code_quality_score`, `logic_correctness_score`, `runtime_efficiency_score`, `overall_score`, `code_capability_score`, `responsiveness_score`, `reliability_score`, `feature_richness_score`, `value_proposition_score` - Performance: `avg_latency_ms`, `p95_latency_ms`, `min_latency_ms`, `max_latency_ms`, `throughput_rps`

---

## 13. Dependencies

**File:** `llm-verifier/go.mod` (172 lines) <sup>[16]</sup>

**Module:** `digital.vasic.llmsverifier`

**Go Version:** `1.25.3`

### Direct Dependencies

| Package                                             | Version               | Purpose                  |
|-----------------------------------------------------|-----------------------|--------------------------|
| <code>github.com/gin-gonic/gin</code>               | <code>v1.11.0</code>  | HTTP web framework       |
| <code>github.com/spf13/cobra</code>                 | <code>v1.10.2</code>  | CLI framework            |
| <code>github.com/spf13/viper</code>                 | <code>v1.21.0</code>  | Configuration management |
| <code>github.com/charmbracelet/bubbletea</code>     | <code>v1.1.0</code>   | TUI framework            |
| <code>github.com/charmbracelet/lipgloss</code>      | <code>v0.13.0</code>  | TUI styling              |
| <code>github.com/mattn/go-sqlite3</code>            | <code>v1.14.32</code> | SQLite driver            |
| <code>github.com/golang-jwt/jwt/v5</code>           | <code>v5.3.0</code>   | JWT authentication       |
| <code>github.com/go-playground/validator/v10</code> | <code>v10.27.0</code> | Input validation         |
| <code>github.com/google/uuid</code>                 | <code>v1.6.0</code>   | UUID generation          |
| <code>github.com/gorilla/websocket</code>           | <code>v1.5.3</code>   | WebSocket support        |
| <code>github.com/stretchr/testify</code>            | <code>v1.11.1</code>  | Testing framework        |
| <code>github.com/swaggo/swag</code>                 | <code>v1.16.6</code>  | Swagger docs             |
| <code>golang.org/x/crypto</code>                    | <code>v0.46.0</code>  | Cryptography             |
| <code>golang.org/x/time</code>                      | <code>v0.14.0</code>  | Rate limiting            |
| <code>cloud.google.com/go/storage</code>            | <code>v1.58.0</code>  | GCS integration          |
|                                                     | <code>v1.6.3</code>   | Azure Blob               |



| Package                                              | Version | Purpose            |
|------------------------------------------------------|---------|--------------------|
| github.com/Azure/azure-sdk-for-go/sdk/storage/azblob |         |                    |
| github.com/aws/aws-sdk-go-v2                         | v1.41.0 | AWS SDK            |
| github.com/minio/minio-go/v7                         | v7.0.98 | S3-compatible      |
| github.com/andybalholm/brotli                        | v1.2.0  | Brotli compression |
| github.com/go-ldap/ldap/v3                           | v3.4.12 | LDAP auth          |
| github.com/quic-go/quic-go                           | v0.54.0 | QUIC/HTTP3         |

## External Module Dependency

`replace digital.vasic.llmprovider => ../../LLMProvider`

**CRITICAL:** The project depends on a local module `digital.vasic.llmprovider` at `../../LLMProvider`. This is a sibling repository that must be present for the project to build.

---

## 14. Test Framework

### Test Directory Structure

**Directory:** `llm-verifier/tests/` <sup>[44]</sup>

| Test Type         | Files                                                                                                                                                         |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unit tests        | <code>tests/unit/</code> , <code>database_unit_test.go</code> , <code>core_components_test.go</code>                                                          |
| Integration tests | <code>tests/integration/</code> , <code>integration_test.go</code> , <code>integration_comprehensive_test.go</code> , <code>integration_simple_test.go</code> |
| E2E tests         | <code>e2e_test.go</code> , <code>acp_e2e_test.go</code>                                                                                                       |
| Performance tests | <code>performance_test.go</code> , <code>acp_performance_test.go</code>                                                                                       |
| Security tests    | <code>security_test.go</code> , <code>acp_security_test.go</code>                                                                                             |
| Automation tests  | <code>automation_test.go</code> , <code>acp_automation_test.go</code>                                                                                         |
| ACP tests         | <code>acp_test.go</code> , <code>acp_integration_test.go</code> , etc.                                                                                        |

## Test Patterns

- Tests use `t.Skip()` with SKIP-OK markers (per CONST-035)
- Mock API server: `tests/mock_api_server.go`
- Test helpers: `tests/test_helpers.go`
- Constants: `tests/test_constants.go`

## Provider Tests

- `providers/adapters_test.go`
- `providers/base_extended_test.go`
- `providers/deepseek_extended_test.go`
- `providers/errors_advanced_test.go`
- `providers/fallback_models_test.go`
- `providers/groq_test.go`
- `providers/integration_test.go`
- `providers/model_provider_service_test.go`

## Scoring Tests

- `scoring/scoring_engine_test.go`
  - `scoring/types_test.go`
  - `scoring/metrics_collector_test.go`
  - `scoring/alert_manager_test.go`
  - `scoring/api_handlers_test.go`
  - `scoring/database_extensions_test.go`
  - `scoring/database_integration_test.go`
  - `scoring/integration_simplified_test.go`
  - `scoring/integration_test.go`
  - `scoring/model_display_test.go`
  - `scoring/model_naming_test.go`
- 

# 15. Integration Examples

## AI CLI Configuration Export

**File:** `llm-verifier/cmd/main.go` (lines 556-599)

```
llm-verifier ai-config export opencode config.json
llm-verifier ai-config bulk ./exports/
llm-verifier ai-config validate config.json
```

## Client Library

**File:** `llm-verifier/client/client.go` — HTTP client with authentication

```
c := client.New(serverURL)
c.Login(username, password)
models, err := c.GetModels()
result, err := c.VerifyModel(modelID)
```

## Docker Support

**File:** `llm-verifier/docker-compose.yml` - SQLite database persistence - Memory caps to prevent host crashes

## Kubernetes

- `helm/llm-verifier/` — Helm chart
  - `k8s/` — Raw Kubernetes manifests
  - `llm-verifier/k8s/` — K8s-specific Go code
- 

# 16. Critical Findings & Gaps for HelixCode Integration

## GAPS — What Needs Extension

- 1. Core Verification is a Stub**  
`verification/verification.go` returns hardcoded scores (all 8.5). The real verification happens in `coding_capability_verification.go` but only covers 4 coding tests. For HelixCode to use LLMsVerifier as a “single source of truth,” the verification engine needs to be completed with actual provider API calls for all dimensions.
- 2. API Server is Minimal**  
`api/server.go` only has 5 basic endpoints. Full CRUD, batch operations, and real-time streaming are likely in other files but not wired in the main server file.
- 3. External Module Dependency**  
`digital.vasic.llmprovider` at `../LLMProvider` is required but not in this repo. HelixCode needs access to the LLMProvider sibling repository.
- 4. No Real-time Push Notifications**  
Only polling and scheduled updates exist. For real-time integration, webhooks or SSE/WebSocket-based push would need to be added.
- 5. Authentication OAuth is Stub-heavy**  
The `auth/oauth_stub.go` files exist to satisfy builds but may not implement full OAuth flows.
- 6. Rate Limiting is Config-only**  
The `limits` table exists but actual enforcement middleware may not be fully implemented in the minimal `api/server.go`.
- 7. Token Pricing Integration**  
Pricing data is stored but there’s no evidence of automatic price fetching from provider APIs. Prices appear to be manually configured or sourced from `models.dev`.
- 8. Scoring Strategy is Extensible**  
`llmverifier/strategy.go`, `strategy_builder.go`, `strategy_default.go` define a Strategy pattern. This is good for HelixCode — custom scoring strategies can be injected.

## STRENGTHS — Ready for Integration

1. **Comprehensive Data Model** — The `models` table and `VerificationResult` struct have ~40+ fields covering all dimensions needed for model evaluation.
  2. **Provider Adapter Pattern** — Clean `BaseAdapter` + per-provider adapters make adding new LLM providers straightforward.
  3. **SQLite + Encryption** — Portable, single-file database with optional SQL Cipher encryption.
  4. **TUI + CLI + REST API** — Three interfaces available; `HelixCode` can use the REST API or embed as a Go library.
  5. **Scoring Engine is Modular** — `ScoringEngine` with configurable weights, batch scoring, and history tracking.
  6. **Report Generation** — Markdown and JSON reports with rankings.
  7. **Configuration Export** — Native support for opencode, crush, claude-code formats.
- 

## 17. Key File Paths Summary

| Area                 | Primary File(s)                                                          |
|----------------------|--------------------------------------------------------------------------|
| CLI Entry            | <code>llm-verifier/cmd/main.go</code>                                    |
| Config Structs       | <code>llm-verifier/config/config.go</code>                               |
| Config Loader        | <code>llm-verifier/llmverifier/config_loader.go</code>                   |
| Core Verifier (stub) | <code>llm-verifier/verification/verification.go</code>                   |
| Coding Verification  | <code>llm-verifier/verification/coding_capability_verification.go</code> |
| Provider Base        | <code>llm-verifier/providers/base.go</code>                              |
| Provider Anthropic   | <code>llm-verifier/providers/anthropic.go</code>                         |
| Provider Config      | <code>llm-verifier/providers/config.go</code>                            |
| Provider Errors      | <code>llm-verifier/providers/errors.go</code>                            |
| LLM Client           | <code>llm-verifier/llmverifier/llm_client.go</code>                      |
| Data Models          | <code>llm-verifier/llmverifier/models.go</code>                          |
| Reporter             | <code>llm-verifier/llmverifier/reporter.go</code>                        |
| Scoring Engine       | <code>llm-verifier/scoring/scoring_engine.go</code>                      |
| Score Types          | <code>llm-verifier/scoring/types.go</code>                               |
| Database             | <code>llm-verifier/database/database.go</code>                           |
| API Server           | <code>llm-verifier/api/server.go</code>                                  |
| Client Lib           | <code>llm-verifier/client/client.go</code>                               |
| TUI                  | <code>llm-verifier/tui/</code>                                           |
| Tests                | <code>llm-verifier/tests/</code>                                         |
| README               | <code>llm-verifier/README.md</code>                                      |
| go.mod               | <code>llm-verifier/go.mod</code>                                         |

| Area        | Primary File(s)          |
|-------------|--------------------------|
| config.yaml | llm-verifier/config.yaml |

---

*Analysis generated from exhaustive repository exploration of <https://github.com/vasic-digital/LLMsVerifier>*